2023 IFIP/IEEE 31st International Conference on Very Large Scale Integration (VLSI-SoC) | 979-8-3503-2599-7/23/$31.00 ©2023 IEEE | DOI: 10.1109/VLSI-SoC57769.2023.10321881

Analyzing the Impact of Different Real Number Formats on the Structural Reliability of TCUs in GPUs

Robert Limas Sierra*, Juan-David Guerrero-Balaguera*, Josie E. Rodriguez Condia*, Matteo Sonza Reorda*
*Politecnico di Torino - Department of Control and Computer Engineering (DAUIN)
{robert.limassierra, juan.guerrero, josie.rodriguez, matteo.sonzareorda}@polito.it

***Abstract*—[1] Modern Graphics Processing Units (GPUs) boost the execution of tiled matrix multiplications by extensively using in-chip accelerators (*Tensor Core Units* or TCUs). Unfortunately, cutting-edge semiconductor technologies are increasingly prone to fault defects. Indeed, faults may affect TCUs when processing massive amounts of data under classical floating-point formats, raising reliability concerns when used in the safety-critical and High-Performance Computing (HPC) domains. In this scenario, the characterization of faulty TCUs supporting different arithmetic formats is still missed. This work for the first time quantitatively evaluates the effects of hardware faults arising in TCU structures when using two different formats for real number representation (i.e., *Floating-Point* and *Posit*). For the experimental evaluation, we resort to an architectural description of a TCU core *(PyOpenTCU)* and perform 60 fault simulation campaigns, injecting 57,344 faults per campaign and requiring around 24 days of computation. The experimental results indicate a relation between the corrupted spatial areas in the output matrices and the TCU's scheduling policies. Moreover, the numeric analysis shows that hardware faults in TCUs in most cases affect up to 2 bits in the output results for both considered formats. The results also demonstrate that the Posit formats are less affected by faults than Floating-Point formats by up to one order of magnitude.**

***Index Terms*—Floating-point numbers, Graphics Processing Units (GPUs), Permanent Faults, Posit numbers, Real number arithmetic, Tensor Core Unit (TCU).**

I. Introduction

Currently, hardware accelerators are essential units that allow to achieve the high computational power demanded by a broad number of modern systems ranging from mobile to high-performance computing (HPC) applications [1]. Most modern applications (e.g., those using Machine Learning algorithms) rely on linear algebra operations, particularly *General Matrix Multiplication* (GEMM or MxM). In fact, algorithms for MxM are intrinsically parallel and can be optimized by resorting to specialized hardware topologies, such as *Systolic Arrays* (SAs) [2], *Dot Product Units* (DPUs) [3] and *in-memory computing* [1].

These topologies are typically incorporated as main engines in application-specific devices like *Tensor Processing Units* (TPUs) in the Artificial Intelligence (AI) domain. Nonetheless, unlike TPUs, Graphic Processing Units (GPUs) evolved as one of the most popular and flexible accelerators in the market and current generations incorporate compacted in-chip MxM accelerators (*Matrix Cores Units*, *Tensor Core Units* or TCUs) to provide mixed-precision computations. The computational flexibility of TCUs makes them suitable for applications beyond the AI, such as in scientific computing that requires high precision, such as cryptography [4], image and video processing [5], and virtual reality [6]. Moreover, safety-critical applications (e.g., autonomous driving, robotics, and healthcare systems) are nowadays pervaded by GPU-based devices, in which the TCU unit plays a fundamental role in conducting advanced and autonomous tasks [7]. Nevertheless, for safety-critical systems reliability represents as a paramount aspect that has to be accomplished by satisfying strict safety standards (e.g., ISO26262 in the automotive domain [8]). Hence, the TCUs inside GPUs must fulfill those reliability criteria as well.

Unfortunately, the astonishing performance of in-chip TCU accelerators of GPUs can be overshadowed by reliability concerns associated with the vulnerabilities of modern semiconductor technologies that make them prone to faults [9], [10]. Permanent faults model possible defects happening due to different situations, such as *i)* test-escaped manufacturing flaws, *ii)* infant mortality phenomena, and *iii)* abrupt damage of the device during in-field operations caused by process variation, premature aging, harsh environment, or high operating temperatures [9]. Furthermore, during the in-field operation of a GPU, permanent faults in TCUs can be eventually activated, and their effects propagate silently to the outputs, leading to catastrophic consequences that endanger the reliability and overall safety of a running application. In this regard, evaluating the impact of permanent faults in TCUs is imperative to identify critical hardware vulnerabilities that allow the devising of innovative hardening strategies during the design phase of the accelerator.

So far, most of the efforts to evaluate the reliability of GEMM accelerators mainly concentrated on assessing the effect of transient faults on different SA topologies [11]–[13]. [14] evaluates the influence of radiation effects in TCUs and mixed-precision formats of GPUs on the overall reliability of MxM operations. The reported experiments indicate that workloads using TCUs have higher failure rates than workloads

[1]This work has been supported by the National Resilience and Recovery Plan (PNRR) through the National Center for HPC, Big Data and Quantum Computing.

979-8-3503-2599-7/23/$31.00 ©2023 IEEE

Authorized licensed use limited to: Politecnico di Torino. Downloaded on October 23,2024 at 15:44:56 UTC from IEEE Xplore. Restrictions apply.

avoiding them. Other works studied the impact of permanent faults of SAs in the context of DNN workloads. Still, these works did not consider the architectural features of TCUs or their internal structures [15]–[17]. Furthermore, most of the related works only assess the resilience of MXM accelerators employing standard format representations (e.g., *Integer* and *Floating-point FP*), and none of them consider accelerators using alternative number formats like the *posit* format for real numbers, so leaving their analyses and evaluations highly unexplored. The *posit* number format is an alternative to tackle weaknesses of IEEE-754 FP format, which might also affect the vulnerability and sensitivity to faults in GEMM accelerators [18].

This work evaluates and analyses the resilience to permanent faults of TCUs in GPUs when two computer formats for real values (*floating-point* and *posit*) are implemented in hardware. We developed an architectural hardware model of TCUs in GPUs (*PyOpenTCU* [2]) together with an embedded fault injector tool to enable reliability evaluations. We evaluated the complete execution cycle of TCUs when permanent faults affect the inputs, outputs, or internal structures in TCUs. The experiments required a total of 60 fault simulation campaigns, injecting 57,344 faults per campaign. Overall, the complete fault simulation campaigns needed around 24 days.

The experimental results indicate a clear relationship among the fault sites, the corrupted areas in the output matrices, the number of TCUs inside each GPU core, and their scheduling policies. Similarly, the numeric analysis showed that hardware faults in TCUs accumulate their effects during the TCU's operative cycle, and in most cases, each fault corrupts 2 bits in the output results for both real number formats. Moreover, the results demonstrate that *Posit* number format is less affected by faults than *Floating-Point* format in up to one order of magnitude. Although, in this paper, we consider the architecture of TCUs in NVIDIA's GPUs, the proposed methodology is likely to apply even to other GPU architectures.

The rest of the paper is organized as follows. Section II provides the related background regarding GPUs, TCUs, and floating point data representations. Section III describes the evaluation methodology. Section IV introduces the framework and experimental setup. Section V presents experimental results for every format representation. Finally, Section VI concludes the paper.

II. Background

A. Organization of Graphics Processing Units (GPUs)

Modern GPUs are massive general-purpose hardware accelerators able to provide high operative throughput by resorting to a set of homogeneous clusters of parallel cores (also known as *Streaming Multiprocessors* or *SMs*). The SM is the primary execution unit in a GPU. It comprises up to four sub-cores to handle and boost the simultaneous execution of several threads (e.g., 32 threads or one *warp*) by resorting to a set of scalar processing cores and in-chip accelerators, such as *Integer units*

[2]https://github.com/TheColombianTeam/PyOpenTCU.git

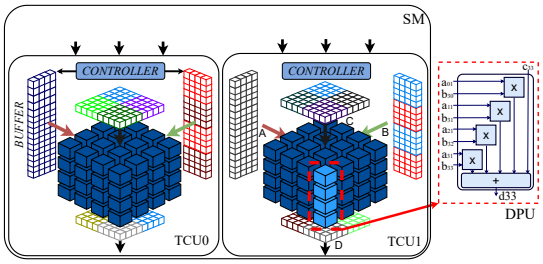

Fig. 1. A general scheme of 2 TCUs inside an SM core. Each TCU comprises 16 DPU cores. Adapted from [3], [19].

(*INT*), *Floating-Point Units* (*FPU*), *Special Function Units* (*SFUs*), and *TCUs*. This variety of resources, in combination with register file banks, memories, and clever scheduling policies, allows the processing of large amounts of data with minimal latency effects. Typically, one SM comprises 32 to 64 INTs and FPUs, 4 SFUs, and 2 TCUs. In particular, the vector nature of TCUs involves special and clever management of the running threads and the memory resources in an SM, also extended into the GPU's ISA capabilities to operate this resource. The following subsection details and highlights the main operative features of TCUs as in-chip accelerators of GPUs.

B. TCU's Organization and Operation in GPUs

In GPUs, TCUs are vector in-chip accelerators boosting the performance during the execution of convolutional and deep learning applications [3], [19]. In detail, a TCU comprises one 4x4 array of *DPU* cores devoted to computing 16 *Multiply-and-Add* (MaA) operations per cycle on matrix segments (e.g., 4x4-size *A*, *B*, and *C* inputs) and perform MxM operations (see Figure 1). Each DPU calculates a MaA operation using inputs from the matrix segments (four operands from A and B and one from C).

Matrix tiling schemes and scheduling policies for parallel hardware [20] distribute the matrix segments (workloads) among the available TCUs. Thread grouping approaches are used to cleverly and efficiently provide data and manage the operation of TCUs. In this case, *Thread groups* (set of 4 threads) allow effective use of TCUs, as well as the sharing of input operands (in their registers). In fact, the scheduling of *Thread groups* into TCUs hides the latency of the loading and storing procedures (e.g., in Figure 1, TCU0 processes *Thread groups* 0, 1, 4, and 5, while TCU1 operates the 2, 3, 6, and 7 *Thread groups*) [3].

TCUs are flexible and support multi-precision and multi-size configurations. To process large matrix dimensions (e.g., 16x16) in TCUs, a sequence of instructions (i.e., *HMMA*) are used (see Figure 2) to allow the operation of 4x4 array-segments. Then, consecutive instructions produce partial, accumulative, and final results. The partial and accumulated results are stored in the register file (or in buffers) for subsequent reuse with other input segments.

C. Numeric Formats for Real Numbers

1) Floating-Point (FP) format: this format encodes a real number into a binary representation code defined by the IEE-

Authorized licensed use limited to: Politecnico di Torino. Downloaded on October 23,2024 at 15:44:56 UTC from IEEE Xplore. Restrictions apply.

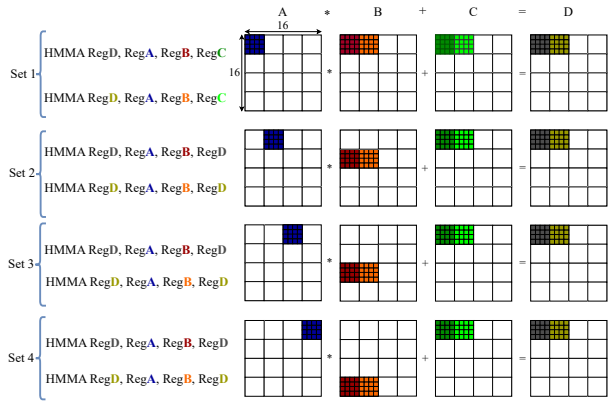

Fig. 2. A general scheme of instructions and input segments used by the *Thread group* 0 to calculate two output segments in the 16x16 mode. The HMMA instructions are grouped in sets to process 4x4 input segments and handle the intermediate results. The result segment D is obtained after the sequence of four sets. The register banks (or buffers) serve as temporal accumulators for partial results. Adapted from [3].

754 standard [21]. This standard establishes a codification using k bits (e.g., 1-bit sign, w-bit exponent, and t-bit fraction). Mathematically, a real value can be described in FP format using equation 1, where $bias$ is a constant offset used to normalize the exponent.

$$value = (-1)^{sign} * 2^{exp-bias} * (1 + \sum_{i=1}^{fn-1} b_{fn-1-i} * 2^{-i}) \quad (1)$$

2) Posit format: this numeric format provides higher accuracy and dynamic range, emerging as an alternative and drop-in replacement for the IEEE-754 format. The *posit* representation encodes any real number into a binary representation using N bits distributed in four fields (i.e., 1-bit sign, r-bit regime, e-bit exponent, and f-bit fraction). This binary encoding describes mathematically any real number according to equation 2.

$$value = (-1)^{sign} * useed^{k} * 2^{exp} * (1 + \sum_{i=1}^{fn-1} b_{fn-1-i} * 2^{-i}) \quad (2)$$

where $useed^k$ is a scale factor $useed = 2^{2^{\mathbf{es}}}$, and **es** is the exponent size. The parameter k is determined by *run length* of 0 or 1 bits in the string of regime bits. The $useed^k$ value is used to compute the min_{pos} and max_{pos} values. Those values determine the Posit dynamic range, maintaining it perfectly balanced at about 1.0, and every *power of 2* has an exact reciprocal. Additionally, the posit format does not include redundancy representations or overflow/underflow cases during operations either. [22].

III. Methodology

This section describes the proposed methodology to evaluate the resiliency of the TCUs inside GPUs. Since our target is the identification of reliability features of the TCUs and the possible effects of permanent faults (i.e., stuck-at-faults) in their architecture and functional operation, we focus our evaluation on the architecture of the TCU cores and main propagation effects on individual scalar values and on the spatial distribution of the results. For this purpose, our methodology comprises 3 steps: *i)* identification of the operational and structural features of TCUs, *ii)* evaluation of the impact of permanent faults in TCUs for two real numeric formats (i.e., *FP* and *Posit*), and *iii)* analysis and classification of fault propagation effects. The following subsections describe the steps for the reliability evaluation of TCUs.

A. Identification of operational features of TCUs.

Since TCUs exploit the buffers (or near-registers) to compensate performance's bottlenecks, the scheduling policies are crucial for the TCU's operations and optimization purposes. In this case, TCU hardware are cleverly reused. Typically, the standard GPU architecture (e.g., NVIDIA Volta) includes 2 TCUs per SM. Thus, all operative threads can address the TCUs, which means that some or all threads might be affected by faulty DPUs inside TCUs. So, the reuse of the TCU hardware is of high interest to understand how the error propagates or gets masked during the execution of MxMs.

Considering that typical data tiles in CNN workloads (e.g., Lenet-5) have a higher amount of zero elements and normalized ranges (e.g., from 0.0 to 1.0). We then decided to select and characterize three types of tiles: *Random, or R* tiles with a random distribution of values, *Zeros, or Z* tiles with numbers close to zero, and *Triangular, or T* tiles with a triangular distribution of values. This selection is also in concordance with other works in the field arguing that fault propagation in the GPU's data path is data independent if the input data (tiles) is not biased (i.e., composed of an excessive amount of 0s or all 1s) [23]

B. Evaluation of permanent faults in TCUs

Evaluating the effects of permanent faults affecting hardware accelerators is crucial for safety-critical applications. However, this kind of analysis requires a great effort, considering the possible structures to be studied depending on the granularity of the unit implementation (e.g., high level, RT level, gate level) and the execution time. Due to this fact, the most feasible option to reduce the simulation time is to resort to architectural models. Using an architectural description represents an effective solution to analyze the faults possibly affecting the different unit structures. Consequently, the fault effects analysis can be based on the equivalent pin-level fault injection. As already said, the TCUs are mainly composed of DPUs. So, understanding the effect of permanent faults on these units is decisive to evaluate the TCU's resiliency. A general architecture scheme of the DPU is presented in Figure 1. Based on this scheme, we identified three critical locations (*inputs IN*, *outputs OUT*, and *product results PR*) inside the DPU, where to inject faults.

When a fault occurs in the *INs*, this fault corresponds to a faulty interconnection in the data path in charge of loading the data to be processed from the registers. Faults in *PRs* model faults in internal units of the multipliers and their error propagation. Finally, if a fault affects any component in the tree adders, it can be modeled at the *OUTs* level. In our

Authorized licensed use limited to: Politecnico di Torino. Downloaded on October 23,2024 at 15:44:56 UTC from IEEE Xplore. Restrictions apply.

work, the resiliency assessment of the TCUs resorts to Fault Simulation (FSim) campaigns. Usually, the Fsim campaign is divided into three main steps: performing the ideal (golden or fault-free) application, injecting the faults, and classifying the results (i.e., comparing each result obtained for the faulty model with the fault-free result) according to the impact of each fault and resorting to the following categories:

- *Masked*: the fault does not produce any effect;
- *Silent Data Corruption* (SDC): the fault affects the results by corrupting one or more output values;
- *Detected Unrecoverable Error* (DUE): the fault prevents the correct execution of the application (i.e., the results show one or more values of *inf* or *NaN*).

Since the TCUs produce results and operate in an accumulative scheme, the final evaluation of the fault effects should be performed only after the accumulative process. In this way, the result used in the classification step corresponds to the final matrix computed.

C. Analysis and classification of fault effects in TCUs

The analysis of FSim campaign results depends on the configuration of the TCU and the numeric formats. In the 16x16 output configuration in the TCU, the output matrix D is computed as a sequence of operations to produce the 256 values. Thus, the resiliency evaluation targets the evaluation of permanent fault effects from two points of view: *i)* individual error effects, and *ii)* spatial effects.

Analyzing the individual numeric effect enables us to compare each value and determine in a quantitative way the permanent fault effect. In order to understand the effect of one permanent fault at the value level, we propose to use the absolute error as an evaluation metric. This metric is based on the discrepancy between the expected value and the value affected by a fault. Moving to the spatial effect, the analysis considers the spatial distribution of the errors in order to identify trends or error patterns. For this reason, this analysis identifies the regions in the output matrix having the highest probability to change their values.

IV. Experimental Setup

To evaluate the resiliency and analyze the effect of permanent faults in TCUs, we developed ***PyOpenTCU***, an architectural model of the operational logic of a TCU operating inside a GPU (thus including DPUs, buffers, and a controller) that can perform the equivalent instruction-wise functional operations of the TCUs inside an SM. PyOpenTCU implements the architecture description presented in [3], [19] and exploits the advantages of a high-level programming language (*Python*) to allow the exploration of different numeric formats, format sizes, and internal organization features, such as the dimension of the spatial arrays of DPUs. Thus, the architectural model provides the flexibility required to study the impact of permanent faults while still demanding feasible evaluation times. Moreover, the architectural model supports the design exploration of different TCU solutions (e.g., changing the numeric format type).

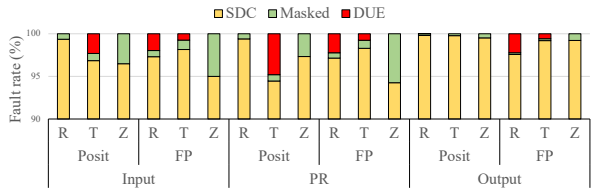

Fig. 3. Fault rate.

For our study case, we explore the scalar effect of two numeric formats for real values (*FP* and *Posit* formats) in TCUs and the effects on their resiliency. Both number formats are supported by *PyOpenTCU* and implemented using the *SoftFloat* and *SoftPosit* [3] libraries.

We also designed a custom environment to automate and speed up the evaluation methodology described in section III. The environment is composed of three main blocks: *i)* a fault injector, *ii)* a fault list generator, and *iii)* a controller.

The *fault injector* is built on top of the PyOpenTCU. This block allows introducing permanent faults in the three mentioned locations of the DPUs in a TCU (*i.e., inputs, outputs, and PRs*) via saboteurs, then allowing to analyze the error propagation. The *fault list generator* produces the list of target faults used in the Fsim campaigns. The *controller* uses the list to activate the saboteur in the DPUs corresponding to each target fault.

Each target fault is composed of a fault descriptor with five configuration parameters (Fault Target - FT, Thread Group ID - TGID, Position - Pos, Mask, and Stuck-at - ST). The TGID and Pos parameters identify the DPU inside the TCU to be affected by the fault. The FT parameter is associated with the target DPU structure to inject a fault (input, output, or PR). The other parameters identify the target bit affected by the fault (Pos and Mask) and the type of stuck-at fault (ST) to apply (0 or 1). The *controller* automates the FSim campaign.

V. Experimental Results

We used the typical configuration of an SM with two TCUs. Thus, in PyOpenTCU, all warps (and their internal *Thread groups*) can address both TCUs. For the evaluation, we use the configuration of 16x16 input matrices (i.e., a complete MxM requires a sequence of 8 HMMA instructions) with a bit-width of 16 for both arithmetic formats. Our FSim campaigns were composed of 60 simulations (30 simulations per considered numeric format, with 10 simulation campaigns per input tile type). Each exhaustive simulation campaign includes a total of 57,344 faults, which are the product of the number of DPUs per TCU (16), the number of inputs per DPU (9), the number of result products per DPU (4), the number of outputs per DPU (1), and the format bit-size (16). This evaluation demanded a computational complexity of 23.89 days, which corresponds to an average of 600 ms per fault. All experiments were performed on a workstation HP Z2 G5 with an Intel Core i9-10800 CPU with 20 cores, and 32 GB of RAM memory.

Firstly, we study the error effects of a fault and its relation with each numeric format. In this case, we compare each

[3] https://pypi.org/project/sfpy/

Authorized licensed use limited to: Politecnico di Torino. Downloaded on October 23,2024 at 15:44:56 UTC from IEEE Xplore. Restrictions apply.

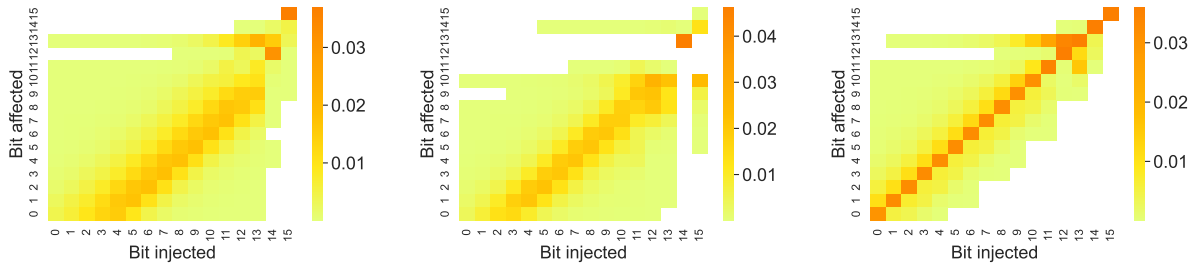

Fig. 4. Trend linking the injected bit-field and the corrupted ones in the result. The picture shows the trend when the input matrices are composed of random values, the TCU computes Posit numbers, and faults are injected in the DPUs' IN (*on left*), and DPUs' OUT (*on the right*) signals. *In the middle* we reported the behavior when the fault affects the DPUs' PR signals and the TCU operates in the FP format.

element of a fault-free matrix with the obtained matrix after the simulation (*affected matrix*). Then, we evaluate the spatial effect of each matrix element and determine the relationship between its output corruption location (i.e., row and column) and the injected fault inside the TCU.

A. Numeric analysis

According to the results, the TCUs are highly sensitive to permanent faults. Figure 3 shows the Fault rate for all study cases. On average, both numeric formats present a similar trend. In detail, 97% of faults injected caused SDCs, the masked cases correspond to 2% of the injected faults, and only 1% of the faults injected are classified as DUE.

A closer look shows that when the fault is located in the DPUs' OUT, the probability of being classified as SDC is 99%. On the other hand, if a fault is located on the IN or PR signals of DPUs, the probability of the fault propagating its impact decreases up to 96%, while the DUE cases increase. Interestingly, the DUE probability is related to the input tile processed. The results show that no faults crash the application when the TCUs compute values close to zero. The DUEs found during this evaluation correspond to cases when the fault affects the most significant (i.e., 13, 14, and 15) bits, regardless of the numeric format. In other words, faults affecting the exponent (in FP) and regime (in Posit) are more prone to crash the application. The masked cases mainly occur when faults corrupt the less significant bits (i.e., 0, 1, and 3 bits), or the more significant bits (i.e., 13, 14, and 15 bits).

Figure 4 shows a heatmap illustrating the relationship between the permanent fault location and the bit affected in the result when the TCUs compute random numbers in both the Posit and FP numeric format. The x-axis shows the bit affected by a fault, the y-axis represents the bit in the matrix output, and the values in the heat map depict the probability of corruption of an output bit.

According to the results, when a fault affects the fraction (in FP) or at the fraction and the exponent (in Posit), it will never affect the exponent and the regime bits. Interestingly, the sign bit only changes the expected value when the fault is located in the most significant bit (14) or in the sign bits, regardless of the numeric format.

The results also show a trend between the bit affected by the fault and the bit changing in the expected result. Since the TCUs are reused several times to compute matrices, the error generated by a fault may accumulate and affect a set

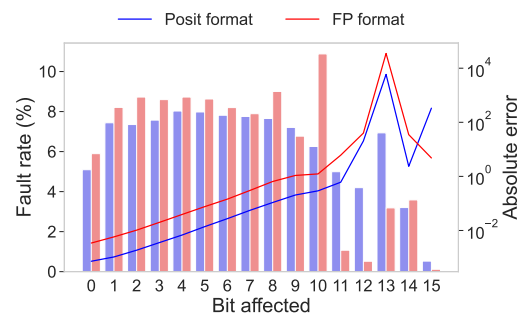

Fig. 5. Fault rate and absolute error per affected bit. In the Posit format, the worst case (*blue*) arises when computing T segments and faults affect the PR in the TCU. In the FP format, the maximum error (*red*) arises for R segments and faults affecting the inputs in the TCU.

of bits. In detail, when the permanent fault affects the IN or PR of the DPUs, there exists a region that can be affected in the final result. The region is a neighborhood centered on the affected bit shifted 2 bits to the left with an area of +/- 3 bits: as an example, if a permanent fault corrupted bit 5, the region with the most probability to change is the bit 3, and those in the area between 0 and 5. In contrast, when the fault affects the DPUs OUT, the corrupted region is directly related: if a fault affects the bit 5, the region with the highest probability to change is also bit 5, as shown in the right part of Figure 4.

The numeric analysis proves that if the TCUs structure presents one permanent fault, there is a 17% probability that that fault affects 2 bits in the expected result. However, the error effect is related to the most significant bit that changes in the final result. As shown in Figure 5, the exponent and the regime bits, in FP and Posit formats, respectively, are more sensitive to permanent fault effects. From our experiments, the absolute error in both formats (FP and Posit) reaches a maximum value of 10^5 and 10^4, respectively. Nevertheless, changes in the exponent (in FP) or in the regime (in Posit) output bits have a probability of less than 14% of occurring. On the other hand, it is more probable that the fraction (in FP) and the fraction and exponent (in Posit) bits present one o more changes. However, the absolute error generated by these changes is less than 1, and in the most common application executed on the TCUs such as the CNN workloads, the magnitude of this error is negligible.

B. Spatial analysis

As mentioned above, our evaluation considered an SM with 2 TCUs (TCU0 and TCU1). Thus, each TCU computes,

Authorized licensed use limited to: Politecnico di Torino. Downloaded on October 23,2024 at 15:44:56 UTC from IEEE Xplore. Restrictions apply.

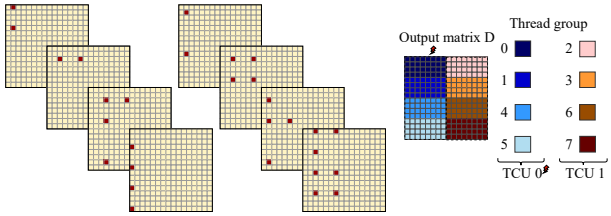

Fig. 6. Spatial distribution of the corruptions in the output matrix from a faulty DPU inside TCU0 (*left*). Distribution of the *Thread groups* in an SM with two TCUs for the 16x16 configuration mode (*right*). Since our injection targeted TCU0, the corrupted elements (from 1 up to 8) per output matrix only appear from the operations processed in TCU0.

in parallel, different segments of the output matrix D. The segment's distribution depends on the scheduling policies (in the GPU) and the tiling schemes used to split them (i.e., the SM re-uses each TCU to compute different segments) [3], [20]. In our experiments, we observed that several output corruption patterns arise in specific areas of D when faults affect the DPU. Thus, the distribution of corrupted elements has a strong relationship with the fault location in the TCU, regardless of the numeric format.

Figure 6 (*left*) shows that the most commonly observed corruption patterns in the output matrix D impact from 2 to 7 elements. Moreover, we observed corruptions from one up to a maximum of 8 corrupted elements per matrix. Interestingly, the location of the corrupted values is related to the mapping of the workload into the DPUs inside the TCU. Thus, a faulty DPU can corrupt up to 8 equally distributed values in the output. In addition, the *Thread group* distribution influences the areas of the result that can be affected by faulty TCUs. The illustrated corruption patterns are produced by faults in TCU0. Nevertheless, the number of locations corrupted can change depending on the type of tile input.

Interestingly, the scheduling policy in the SMs for the TCUs and the internal mapping of DPUs seem to play an important role in terms of the impact of faults on the output results. The identification of the spatial error distribution serves as starting point for the high-level evaluation of applications using TCU cores, as well as the definition of the error model for TCUs.

VI. Conclusions

This work analyzes the TCUs' resiliency inside the GPUs. We introduced a flexible architectural model named *Py-OpenTCU* that allows the analysis and evaluation of the architectural impacts of permanent faults affecting TCUs.

The experiments show that the exponent (in FP format) and the regime (in Posit format) fields are highly sensitive to faults. The results indicate that the FP format is more sensitive to permanent faults than the Posit one by up to one order of magnitude.

The results also show that 2 corrupted bits in the output result are the most frequent effect of permanent faults in TCUs. Interestingly, the most significant corrupted bit produces a higher error effect. Moreover, the results indicate a strong relationship between the spatial corruption in the output and the scheduling policies. According to the organization of an SM and the number of TCUs, the corruption regions might change, so indicating the SM's sensitivity to its structural organization.

References

[1] B. Peccerillo *et al.*, "A survey on hardware accelerators: Taxonomy, trends, challenges, and perspectives," *Journal of Systems Architecture*, vol. 129, p. 102561, 2022.

[2] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," *SIGARCH Comput. Archit. News*, vol. 45, no. 2, p. 1–12, 2017.

[3] M. Raihan *et al.*, "Modeling deep learning accelerator enabled gpus," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, mar 2019, pp. 79–92.

[4] W.-K. Lee *et al.*, "Tensorcrypto: High throughput acceleration of lattice-based cryptography using tensor core on gpu," *IEEE Access*, vol. 10, pp. 20 616–20 632, 2022.

[5] S. Groth *et al.*, "Efficient application of tensor core units for convolving images." Association for Computing Machinery, 2021.

[6] T. Oakden *et al.*, "Graphics processing in virtual production," in *2022 14th International Conference on Computer and Automation Engineering (ICCAE)*, 2022, pp. 61–64.

[7] A. Lee. (2022) Train spotting: Startup gets on track with ai and nvidia jetson to ensure safety, cost savings for railways. [Online]. Available: https://resources.nvidia.com/en-us-jetson-success/rail-vision-startup-uses?lx=XRDs_y

[8] R. Mariani. (2023) Driving toward a safer future: Nvidia achieves safety milestones with drive hyperion autonomous vehicle platform. [Online]. Available: https://blogs.nvidia.com/blog/2023/04/20/nvidia-drive-safety-milestones/

[9] IEEE, "The international roadmap for devices and systems: 2022," in *Institute of Electrical and Electronics Engineers (IEEE)*, 2022.

[10] A. J. Strojwas *et al.*, "Yield and reliability challenges at 7nm and below," in *2019 Electron Devices Technology and Manufacturing Conference (EDTM)*, 2019, pp. 179–181.

[11] F. Libano *et al.*, "On the reliability of xilinx's deep processing unit and systolic arrays for matrix multiplication," in *2020 20th European Conference on Radiation and Its Effects on Components and Systems (RADECS)*, 2020, pp. 1–5.

[12] P. Omland *et al.*, "Hpc hardware design reliability benchmarking with hdfit," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 3, pp. 995–1006, 2023.

[13] R. L. Rech and P. Rech, "Reliability of google's tensor processing units for embedded applications," in *2022 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2022, pp. 376–381.

[14] P. M. Basso *et al.*, "Impact of tensor cores and mixed precision on the reliability of matrix multiplication in gpus," *IEEE Transactions on Nuclear Science*, vol. 67, no. 7, pp. 1560–1565, 2020.

[15] S. Kundu *et al.*, "Special session: Reliability analysis for ai/ml hardware," in *2021 IEEE 39th VLSI Test Symposium (VTS)*, 2021, pp. 1–10.

[16] E. Ozen and A. Orailoglu, "Architecting decentralization and customizability in dnn accelerators for hardware defect adaptation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 3934–3945, 2022.

[17] A. Chaudhuri *et al.*, "Special session: Fault criticality assessment in ai accelerators," in *2022 IEEE 40th VLSI Test Symposium (VTS)*, 2022, pp. 1–4.

[18] J. L. Gustafson, "Beyond floating point: Next-generation computer arithmetic," in *Stanford Talk*, 2017.

[19] B. R. Boswell *et al.*, "Generalized acceleration of matrix multiply accumulate operations," Jul. 2 2019, u.S. Patent and Trademark Office, US Patent 10,338,919.

[20] J. Huang, C. D. Yu, and R. A. van de Geijn, "Implementing strassen's algorithm with cutlass on nvidia volta gpus," 2018.

[21] "Ieee standard for floating-point arithmetic," *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, 2019.

[22] Gustafson and Yonemoto, "Beating floating point at its own game: Posit arithmetic," *Supercomput. Front. Innov.: Int. J.*, vol. 4, no. 2, p. 71–86, jun 2017.

[23] F. G. Previlon *et al.*, "A comprehensive evaluation of the effects of input data on the resilience of gpu applications," in *2019 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, 2019.

Authorized licensed use limited to: Politecnico di Torino. Downloaded on October 23,2024 at 15:44:56 UTC from IEEE Xplore. Restrictions apply.